

팀스파르타 제공 · 원본 유지

PART 1. 클라우드 인프라

서버·스토리지·네트워크 같은 자원을 직접 구축하지 않고 AWS·GCP·Azure 같은 사업자에게서 필요한 만큼 빌려 쓰는 구조.

스토리지, 네트워킹, 데이터베이스가 핵심 자원이고, 컨테이너(Docker)·오케스트레이션(Kubernetes)으로 배포하며 트래픽에 따라 자동으로 확장

=> 보안·확장성 관리는 사용자가 직접 설계해야 함.

학습 목표

- On-prem / Cloud / Hybrid 인프라의 차이를 설명할 수 있다
- Docker와 Kubernetes가 LLM 서빙에서 하는 역할을 이해한다
- IaC 기반 인프라 구성의 필요성을 설명할 수 있다

도입

"Private LLM을 서비스에 도입한다고 하면, 가장 먼저 부딪히는 질문이 하나 있습니다.

'이 모델을 어디에 올릴 것인가'입니다.

우리 회사 서버실에 GPU를 직접 사서 놓을 것인지, 클라우드에서 GPU를 빌려 쓸 것인지에 따라 비용 구조와 데이터 보안 수준이 완전히 달라집니다.

오늘은 이 선택지들을 비교하고, 그 위에서 실제로 어떻게 서버를 구성하고 관리하는지까지 살펴보겠습니다."

1-1 인프라 유형 비교

Private LLM을 어디에 올릴지 결정하는 첫 단계는 인프라 유형을 정하는 것이다.

유형	정의	특징
On-prem	자체 서버실·데이터센터에 직접 GPU를 구축	초기 투자는 크지만 데이터 통제권이 높음
Cloud	AWS, GCP, Azure 등 클라우드 GPU 인스턴스 활용	초기 비용이 낮고 확장이 빠르지만 사용량에 따라 비용 증가
Hybrid	On-prem과 Cloud를 병행	민감 데이터는 On-prem, 트래픽 급증 시 Cloud로 확장(버스팅)

비유로 설명하기: On-prem은 집을 사는 것,

Cloud는 집을 빌리는 것,

Hybrid는 평소엔 자가에서 지내다 손님이 몰리는 성수기에만 근처 숙소를 추가로 빌리는 것에 가깝다.

집을 사면 초기 비용은 크지만 내 마음대로 쓸 수 있고, 빌리면 부담 없이 시작할 수 있지만 매달 비용이 나간다.

확인 질문: "만약 여러분이 병원의 진료 기록을 다루는 LLM 서비스를 만든다면, 어떤 유형을 선택 하시겠습니까? 왜 그렇게 생각하시나요?"

답변 방향: 데이터 민감도가 높은 도메인일수록 On-prem이나 Hybrid를 우선 고려하게 된다

키워드 명사부터 알자

On-prem

On-premises(온프레미스)의 약자 = 'Premises'는 건물, 구내, 사옥.

기업이 자체 전산실이나 사옥 내 서버룸에 하드웨어를 직접 설치해 운영하는 방식

(예시) 클라우드(Cloud): '아파트 월세/호텔 장기 투숙' vs 온프레미스(On-prem): '내가 직접 지은 단독주택'

1-2 Docker와 Kubernetes

구성요소	역할
Docker	서빙 환경 패키징 및 재현성 확보
Kubernetes	GPU 자원 배치, 확장, 복구 자동화

Docker

모델·라이브러리·의존성을 하나의 이미지로 묶어, 어디서나 동일하게 실행되도록 만드는 컨테이너 기반의 가상화 오픈소스 플랫폼.

OS의 핵심 자원만 공유하면서 "내 컴퓨터에서는 잘 돌아가는데 서버에서는 왜 안 되지?"라는 환경 차이 문제가 완전히 사라짐

Kubernetes

여러 대의 GPU 서버를 하나의 클러스터로 묶어, 트래픽에 따라 서빙 파드를 늘리거나 줄이고 장애가 난 파드를 자동으로 복구한다.

자동 배치(스케줄링), 자동 복구(자가 치유), 오토 스케일링(트래픽 대응)

비유로 설명하기: 도커(Docker): '표준 규격 화물 컨테이너' = 배, 기차, 트럭 어디에 실어도 규격이 딱 맞아 떨어지는 해운 컨테이너.

쿠버네티스(Kubernetes): '초대형 항만 물류 터미널의 총괄 관제 시스템' = 수백, 수천 개의 컨테이너를 자동으로 배치하고 조율하는 '전체 물류 시스템(오케스트레이션)'.

확인 질문: "트래픽이 갑자기 10배로 늘어났을 때, Docker만 있고 Kubernetes가 없다면 어떤 문제가 생길까요?"

답변 방향. 서버를 늘리는 작업을 사람이 수동으로 해야 해서 대응이 늦어진다

두 기술이 함께 쓰이는 방식

둘은 경쟁 관계가 아니라 상호 보완적인 파트너

빌드 단계 (Docker 담당):

- 개발자가 스프링 부트 웹 애플리케이션 코드를 작성한 뒤, Dockerfile을 사용해 `도커 이미지`로 만듭니다.

배포 및 운영 단계 (Kubernetes 담당):

- 쿠버네티스 설정 파일(YAML)에 "방금 만든 도커 이미지를 최소 5개 띄우고, CPU 사용률이 70%를 넘으면 10개까지 늘려줘"라고 지시합니다.
- 쿠버네티스는 분산된 여러 서버 장비(노드)에 컨테이너들을 나누어 배치하고, 트래픽을 분산시키며 24시간 감시합니다.

키워드 명사부터 알자

Docker 항만 노동자

Kubernetes 조타수(배의 방향을 잡는 항해사) ,

K8s 줄임말 이유 'K'와 's' 사이에 철자가 8개(ubernete) 들어가 있어서 앞뒤 글자를 따 **K8s**라고 부름

1-3 IaC — Infrastructure as Code

IaC는 수동으로 서버나 네트워크를 설정하는 대신, 코드를 작성해 인프라를 생성하고 관리하는 기술이다.

IaC를 쓰면 인프라 구성이 코드로 남기 때문에, 동일한 환경을 반복해서 재현하거나 변경 이력을 추적하기 쉬워진다.

도구	담당 영역
----	-------

Terraform	클러스터, 네트워크, GPU 노드풀 등 인프라 구성
-----------	------------------------------

Ansible	드라이버, CUDA, 에이전트 등 서버 설정 자동화
---------	------------------------------

Terraform 클라우드(AWS, GCP, Azure) 및 온프레미스 장비를 포괄하는 가장 대표적인 IaC 도구. 확장 설치후 VS-CODE 터미널에서 사용가능 (예) terraform init, terraform plan

Ansible 서버 내부의 소프트웨어 설치 및 환경 설정 자동화에 특화. 확장 설치후 VS-CODE 터미널에서 사용가능 (예) YAML 파일로 작성

비유로 설명하기: - 기존 수동 방식: '현장 수제 가구 제작' = 똑같은 책상 100개를 만들려면 100번의 고된 수작업을 반복

- IaC 방식: '3D 프린터용 디지털 설계도(CAD 파일)' = 컴퓨터에 완벽한 치수와 재질이 적힌 코드 (설계도)를 한 번 작성 => 언제든지 동일한 결과물을 재현

키워드 명사부터 알자

IaC (Infrastructure as Code, 코드형 인프라) :

서버, 네트워크, 스토리지 등 IT 인프라를 소프트웨어 코드를 작성해 자동으로 생성·배포·관리하는 기술

정리

"오늘 살펴본 내용을 정리하면, Private LLM 인프라를 구성할 때는 먼저 On-prem/Cloud/Hybrid 중 어디에 올릴지 정하고, 그 위에서 Docker로 서빙 환경을 포장하고 Kubernetes로 여러 서버를 자동으로 운영하며, 이 모든 과정을 IaC로 코드화해서 반복 가능하게 만든다는 흐름이었습니다."

다음 파트 예고: "이렇게 인프라를 갖췄다면, 이제 그 위에서 모델이 실제로 요청을 처리할 때 무엇이 느껴지는지, 서빙 관점의 이야기로 넘어가겠습니다."